

to our method. However, it does not use attention mechanism, and by having fixed sized softmax output over the relative pointing range (e.g., -7, ..., -1, 0, 1, ..., 7), their model (the Positional All model) has a limitation in applying to more general problems such as summarization and question answering, where, unlike machine translation, the length of the context and the pointing locations in the context can vary dramatically. In question answering setting, (Hermann et al., 2015) have used placeholders on named entities in the context. However, the placeholder id is directly predicted in the softmax output rather than predicting its location in the context.

The third category of the approaches changes the unit of input/output itself from words to a smaller resolution such as characters (Graves, 2013) or bytecodes (Sennrich et al., 2015; Gillick et al., 2015). Although this approach has the main advantage that it could suffer less from the rare/unknown word problem, the training usually becomes much harder because the length of sequences significantly increases.

Simultaneously to our work, (Gu et al., 2016) and (Cheng and Lapata, 2016) proposed models that learn to copy from source to target and both papers analyzed their models on summarization tasks.

3 Neural Machine Translation Model with Attention

As the baseline neural machine translation system, we use the model proposed by (Bahdanau et al., 2014) that learns to (soft-)align and translate jointly. We refer this model as NMT.

The encoder of the NMT is a bidirectional RNN (Schuster and Paliwal, 1997). The forward RNN reads input sequence $\mathbf{x} = (x_1, \dots, x_T)$ in left-to-right direction, resulting in a sequence of hidden states $(\vec{\mathbf{h}}_1, \dots, \vec{\mathbf{h}}_T)$. The backward RNN reads $\mathbf{x}$ in the reversed direction and outputs $(\overleftarrow{\mathbf{h}}_1, \dots, \overleftarrow{\mathbf{h}}_T)$. We then concatenate the hidden states of forward and backward RNNs at each time step and obtain a sequence of *annotation* vectors $(\mathbf{h}_1, \dots, \mathbf{h}_T)$ where $\mathbf{h}_j = [\vec{\mathbf{h}}_j || \overleftarrow{\mathbf{h}}_j]$. Here, $||$ denotes the concatenation operator. Thus, each annotation vector $\mathbf{h}_j$ encodes information about the j -th word with respect to all the other surrounding words in both directions.

In the decoder, we usually use gated recurrent unit (GRU) (Cho et al., 2014; Chung et al.,

2014). Specifically, at each time-step t , the soft-alignment mechanism first computes the relevance weight e_{tj} which determines the contribution of annotation vector $\mathbf{h}_j$ to the t -th target word. We use a non-linear mapping f (e.g., MLP) which takes $\mathbf{h}_j$, the previous decoder's hidden state $\mathbf{s}_{t-1}$ and the previous output y_{t-1} as input:

$$e_{tj} = f(\mathbf{s}_{t-1}, \mathbf{h}_j, y_{t-1}).$$

The outputs e_{tj} are then normalized as follows:

$$l_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^T \exp(e_{tk})}. \quad (1)$$

We call l_{tj} as the relevance score, or the alignment weight, of the j -th annotation vector.

The relevance scores are used to get the *context vector* $\mathbf{c}_t$ of the t -th target word in the translation:

$$\mathbf{c}_t = \sum_{j=1}^T l_{tj} \mathbf{h}_j,$$

The hidden state of the decoder $\mathbf{s}_t$ is computed based on the previous hidden state $\mathbf{s}_{t-1}$, the context vector $\mathbf{c}_t$ and the output word of the previous time-step y_{t-1} :

$$\mathbf{s}_t = f_r(\mathbf{s}_{t-1}, y_{t-1}, \mathbf{c}_t), \quad (2)$$

where f_r is GRU.

We use a deep output layer (Pascanu et al., 2013) to compute the conditional distribution over words:

$$p(y_t = a | y_{<t}, \mathbf{x}) \propto \exp \left(\psi_{(\mathbf{W}_o, \mathbf{b}_o)}^a f_o(\mathbf{s}_t, y_{t-1}, \mathbf{c}_t) \right), \quad (3)$$

where $\mathbf{W}$ is a learned weight matrix and $\mathbf{b}$ is a bias of the output layer. f_o is a single-layer feed-forward neural network. $\psi_{(\mathbf{W}_o, \mathbf{b}_o)}(\cdot)$ is a function that performs an affine transformation on its input. And the superscript a in ψ^a indicates the a -th column vector of ψ .

The whole model, including both the encoder and the decoder, is jointly trained to maximize the (conditional) log-likelihood of target sequences given input sequences, where the training corpus is a set of $(\mathbf{x}_n, \mathbf{y}_n)$'s. Figure 2 illustrates the architecture of the NMT.